Rompicapo delle 8 regine

20/12/2022

Studente: Emanuele Izzo *Matricola: 0307385*

Indice

1 Gli scacchi **1**

2 Definizione del problema **3**

3 Implementazioni **4**

3.1 Implementazione in Python 4
3.1.1 Controllo della colonna 5
3.1.2 Controllo della riga 5
3.1.3 Controllo delle diagonali 5
3.1.4 Controllo della soluzione 5
3.1.5 Copia della soluzione 6
3.1.6 Stampa della soluzione 6
3.1.7 Generazione delle soluzioni 6
3.1.8 Esempio di esecuzione 8
3.2 Implementazione in Prolog 8
3.2.1 Controllo della coppia 8
3.2.2 Controllo della diagonale 9
3.2.3 Controllo delle coppie di regine 9
3.2.4 Controllo della soluzione 9
3.2.5 Esempio di esecuzione 9

1 Gli scacchi

Gli scacchi sono un gioco di strategia che si svolge su una tavola quadrata detta scacchiera, formata da 64 caselle (o ”case”) di due colori alternati, sulla quale ogni giocatore dispone di 16 pezzi (bianchi o neri, i quali designano i due sfidanti[1]): un re, una donna o regina, due alfieri, due cavalli, due torri e otto pedoni; ogni casella può essere occupata da un solo pezzo, che può catturare o ”mangiare” il pezzo avversario andando a occuparne la casella; obiettivo del gioco è dare scacco matto, ovvero minacciare la cattura del re avversario in modo tale che l'altro



Figura 1: Scacchiera e posizione dei pezzi

[1]Bisogna appuntare che gli scacchi non sono necessariamente bianchi e neri; a decidere il colore attribuito a ogni giocatore è la disposizione iniziale dei pezzi (il Bianco ha la donna alla sinistra del re, il Nero alla sua destra), la quale determina a chi spetterà la prima mossa (per convenzione, apre il Bianco).

giocatore non abbia mosse legali. Ciascun pezzo degli scacchi si muove con precise modalità. Nessun pezzo può andare a occupare una casa in cui è presente un altro pezzo dello stesso schieramento; può invece muoversi su una casa occupata da un pezzo avversario, effettuando in tal caso una ”cattura”, cioè eliminando (”mangiando”) dalla scacchiera il pezzo nemico e prendendo il suo posto. Si dice che un pezzo ”attacca” o ”minaccia” una casa se esso può muoversi su di essa. I movimenti che possono effettuare le pedine sono i seguenti:

- L'alfiere può muoversi su una qualunque casa della stessa diagonale rispetto a quella in cui si trova, purché per raggiungerla non debba attraversare case occupate da pezzi (amici o avversari) e purché la casa d'arrivo non sia occupata da un pezzo amico. Ciascun alfiere non cambia mai il colore delle case su cui si muove: per questo i giocatori parlano di alfieri ”campochiaro” o ”camposcuro” a seconda del colore delle case in cui si trovano.

- La torre può muoversi su una qualunque casa della stessa traversa o della stessa colonna rispetto a quella in cui si trova, purché per raggiungerla non debba attraversare case occupate da pezzi (amici o avversari) e purché la casa d'arrivo non sia occupata da un pezzo amico.

- La donna è il pezzo più potente di tutti e combina le mosse dell'alfiere con quelle della torre, potendo quindi muoversi su tutte le case della stessa traversa, della stessa colonna o della stessa diagonale della casa su cui si trova.

- Il cavallo può muoversi su una delle case a lui più vicine che non appartengono alla traversa, alla colonna e alle diagonali passanti per la sua casa di partenza. Un cavallo al centro della scacchiera ha a disposizione otto case (”rosa di cavallo”) verso le quali muoversi, mentre se si trova al bordo la sua mobilità è ridotta a quattro o tre case e se si trova in un angolo a due. Il movimento del cavallo può essere immaginato come la somma di uno spostamento orizzontale di una casa e di uno verticale di due o viceversa, descrivendo una ”L”. Il cavallo è inoltre l'unico pezzo che, nei suoi movimenti, può attraversare anche caselle già occupate da altri pezzi: si dice che può ”saltare”. Si noti che a ogni mossa il cavallo cambia il colore della casa in cui si trova.

- Il pedone segue regole di movimento leggermente più complesse:

  - Alla sua prima mossa, ossia quando parte dalla posizione iniziale, ciascun pedone può muovere di una oppure due case in avanti, a scelta del giocatore, a patto che la casa di destinazione ed eventualmente la casa saltata siano libere. Nelle sue mosse successive il pedone può avanzare solo di una casa per mossa, a patto che questa sia libera. A differenza degli altri pezzi, non può muovere all'indietro.
  - Il pedone è l'unico pezzo che cattura in maniera differente da come muove. Può catturare un pezzo nemico solo se si trova su una delle due case poste diagonalmente in avanti rispetto alla sua casa di partenza, ma non può né muovere in tali case se esse sono libere né catturare i pezzi che si trovano nelle case che ha di fronte.
  - Quando, eseguendo la sua prima mossa di due case in avanti, il pedone viene a trovarsi di fianco a un pedone avversario, quest'ultimo può alla mossa successiva catturarlo *en passant*, come se il primo fosse avanzato di una sola casa. La presa en passant può

essere eseguita solo come mossa successiva all'avanzamento del pedone avversario di due case.

  - Se un pedone riesce ad avanzare fino all'ottava traversa, viene promosso, ossia assume il ruolo e le capacità di movimento di un altro pezzo dello stesso colore (donna, torre, alfiere o cavallo) a scelta del giocatore, indipendentemente dai pezzi già presenti sulla scacchiera. In questo modo è dunque possibile avere un numero di esemplari di un certo pezzo maggiore rispetto a quello iniziale. L'effetto è immediato, ad esempio si può dare scacco o scacco matto con una promozione se il re avversario è nel raggio di azione del nuovo pezzo.

- Il re si può muovere in una delle case adiacenti (anche diagonalmente) a quella occupata, purché questa non sia controllata da un pezzo avversario.

Il re è l'unico pezzo che non viene mai catturato, ma solo minacciato. Quando il re di uno dei due giocatori è minacciato, trovandosi sulla traiettoria di un pezzo nemico, si dice che è "sotto scacco" e non è consentita alcuna mossa che lasci il proprio re in tale condizione. Deve quindi essere effettuata una mossa che elimini la minaccia in uno dei seguenti modi:

- Muovere il re in una delle case adiacenti, a patto che questa non sia sotto il controllo di un pezzo avversario;

- Catturare, con il re o con un altro pezzo, il pezzo avversario che si trova sulla traiettoria del re e dà origine allo scacco;

- Nel caso di minaccia da parte di donna, torre o alfiere non adiacenti al re sotto attacco, frapporre tra quest'ultimo e il pezzo che minaccia scacco un qualunque pezzo o pedone, in modo che sia quest'ultimo a essere minacciato invece del re.

Se nessuna delle mosse che il giocatore può effettuare è in grado di liberare il re dallo scacco, si tratta di scacco matto e la partita termina con la vittoria dell'avversario. Se invece il re non si trova sotto scacco ma non è possibile effettuare alcuna mossa legale (ad esempio se si ha solo il re in gioco e questo non è sotto scacco, ma tutte le case libere a esso adiacenti sono minacciate), si tratta di stallo e la partita termina con un risultato di parità, non potendo il giocatore che si trova in questa condizione muovere senza contravvenire a qualche regola del gioco.

2 Definizione del problema

Il rompicapo delle 8 regine è un problema proposto nel 1848 dallo scacchista tedesco Max Bezzei. Il problema è definito nel seguente modo:

Definizione 2.1. Data una scacchiera $8 \cdot 8$, posizionare 8 regine in modo tale che nessuna delle regine possa mangiarne altre tramite una sola mossa.

È possibile definire una versione generalizzata del problema per scacchiere di varia grandezza:

Definizione 2.2. Sia $k \geq 4$, data una scacchiera $k \cdot k$, posizionare k regine in modo tale che nessuna delle regine possa mangiarne altre tramite una sola mossa.

Possiamo definire tale problema in un altro modo ragionando in questo modo:

- Una scacchiera di dimensione $k \cdot k$ può essere vista come una griglia $\mathbb{S} := [1, k]^2$.
- Utilizzando tale rappresentazione, la posizione di ogni pezzo può essere rappresentata tramite delle coordinate $(x, y) \in \mathbb{S}$.

Per comodità non andiamo a considerare come distinguere univocamente i vari pezzi sulla scacchiera, poiché andremo a lavorare solo con le regine. A prescindere da ciò, possiamo ridefinire il problema come segue:

Definizione 2.3. Sia $k \geq 4$, data una griglia $S := [1, k]^2$, posizionare k punti $q_i = (x_i, y_i)$, con $i \in [1, k]$, in modo che, per ogni coppia $i, j \in [1, k]$ tale che $i \neq j$, abbiamo che:

- $x_i \neq x_j$.
- $y_i \neq y_j$.
- Non esiste alcun $\delta \in [1, k]$ tale che $x_i = x_j + \delta$ e $y_i = y_j + \delta$.

Quest'ultima definizione del problema ci sarà utile nel definire un sistema per generare tutte le possibili soluzioni. Prima di procedere è giusto spendere due parole sul perché $k \geq 4$. Prendendo $k < 4$, abbiamo solo 3 possibili casi:

- Per $k = 1$ il problema è risolvibile ma banale.
- Per $k = 2$ e $k = 3$ il problema non è risolvibile (a prescindere come posizioniamo le regine, saranno in grado di mangiarsi a vicenda con una sola mossa).

3 Implementazioni

3.1 Implementazione in Python

L'implementazione riportata di seguito si divide nei seguenti punti:

- Per prima cosa andremo a definire i vari controlli sulle condizioni di validità di una soluzione.
- Successivamente andremo a definire la funzione per il controllo complessivo della soluzione.

Verranno definite inoltre due funzioni di utility: una per la stampa e una per la copia della soluzione. Una prima cosa che sarà necessario definire è un valore per rappresentare sia la grandezza della scacchiera, sia il numero di regine con cui lavoriamo (in questo esempio viene settato ad 8).

```
1 GRANDEZZA = 8
```

3.1.1 Controllo della colonna

La funzione controlloCol(q_i, q_j) prende le coordinate di due regine e restituisce semplicemente vero se le due regine non sono sulla stessa colonna, o falso altrimenti.

```
# controlloCol(q_i, q_j) => boolean
# Parametri:
# - q_i -> Posizione della regina i (é un'array composto da 2 elementi)
# - q_j -> Posizione della regina j (é un'array composto da 2 elementi)
```

3.1.2 Controllo della riga

La funzione controlloRiga(q_i, q_j) prende le coordinate di due regine e restituisce semplicemente vero se le due regine non sono sulla stessa riga, o falso altrimenti.

```
# - Restituisce vero se le due regine non sono sulla stessa colonna
# - Restituisce falso altrimenti
def controlloCol(q_i, q_j):
  if q_i[1] == q_j[1]:
```

3.1.3 Controllo delle diagonali

Per il controllo delle diagonali possiamo usare la definizione della condizione vista sopra: preso un $\delta \in [1, k]$, una soluzione valida ha che per ogni coppia di regine diverse $q_i = (x_i, y_i)$ e $q_j = (x_j, y_j)$ abbiamo $|x_i - x_j| \neq |y_i - y_j|$. Quello che possiamo fare è calcolarci quindi i valori $|x_i - x_j|$ e $|y_i - y_j|$, e verificare se questi valori siano uguali oppure no: nel caso siano equivalenti si restituirà falso, altrimenti si restituirà vero.

```
  return True

# controlloRiga(q_i, q_j) => boolean
# Parametri:
# - q_i -> Posizione della regina i (é un'array composto da 2 elementi)
# - q_j -> Posizione della regina j (é un'array composto da 2 elementi)
```

3.1.4 Controllo della soluzione

Il controllo della soluzione è molto semplice: preso l'insieme delle regine, e prese le regine a coppie in questo insieme, si verifica che le due regine non siano uguali, non siano sulla stessa riga, sulla stessa colonna e sulle stesse diagonali.

```
# - Restituisce vero se le due regine non sono sulla stessa riga
# - Restituisce falso altrimenti
def controlloRiga(q_i, q_j):
  if q_i[0] == q_j[0]:
    return False
  return True
```

```
7
8  # controlloDiag(q_i, q_j) => boolean
```

3.1.5 Copia della soluzione

La copia della soluzione avviene in un modo molto semplice: vengono prese tutte le posizioni delle varie regine, e vengono poi copiate in una lista vuota che è stata aggiunta precedentemente ad S, il nostro insieme delle soluzioni generate (S è una matrice definita come segue: la riga definisce la soluzione, la colonna la regina di una determinata soluzione).

```
1  def copiaSoluzione(Q, S):
2      S.append([])
3      i = len(S)-1
4      for q in Q:
5          S[i].append(q)
```

3.1.6 Stampa della soluzione

Questa principalmente è una funzione di utility, usata per rappresentare graficamente le varie soluzioni e semplificarne la lettura.

```
1  def stampa(Q):
2      C = [[" " for i in range(GRANDEZZA)] for j in range(GRANDEZZA)]
3      i = 0
4      for q in Q:
5          C[q[0]][q[1]] = "Q"
6      for j in range(GRANDEZZA*2+1):
7          if j % 2 == 0:
8              for k in range(GRANDEZZA):
9                  print("+-", end = '')
10             print("+")
11         else:
12             for k in range(GRANDEZZA):
13                 print(f"|{C[i][k]}", end = '')
14             print("|")
15             i += 1
```

3.1.7 Generazione delle soluzioni

La generalizzazione delle soluzioni viene fatta utilizzando la funzione ricorsiva generaSoluzioni (Q, i, S), la quale prende come parametri la lista delle regine piazzate, l'indice della regina che si deve posizionare e la lista attuale di soluzioni generate; una particolarità di questa funzione è il fatto che il controllo della soluzione viene effettuato ogni volta che viene posizionata una regina, e non solo quando tutte le regine sono state posizionate; questo permette di eliminare da subito tutte le soluzioni che già nel posizionamento delle prime regine risultano invalide. Per dare il via alla

generazione delle soluzioni viene chiamata la funzione trovaSoluzioni (), che restituisce la matrice S delle soluzioni.

```
# generaSoluzioni(Q, S)
# Parametri:
# - Q -> Insieme delle posizioni delle regine
#        (é una tabella dove ogni riga rappresenta una regina,
#        mentre ogni colonna rappresenta una delle due coordinate)
# - i -> Indice della riga in cui inserire la regina
# - S -> Insieme delle soluzioni corrette
#        (é un'array tridimensionale definito in questo modo:
#        la prima dimensione definisce la soluzione considerata,
#        la seconda dimensione la regina della soluzione considerata,
#        la terza dimensione le coordinate della regina della soluzione
    considerata)
# Procedimento:
# - Verifica se il valore i é uguale alla variabile grandezza: In caso
   positivo, verifica che la posizione delle regine genera una
   soluzione valida e in caso positivo la salvo, altrimenti per ognuna
   delle celle della riga sottostante richiamo ricorsivamente il
   meotodo,
def generaSoluzioni(Q, i, S):
  if i == GRANDEZZA:
    if controlloSoluzione(Q):
      copiaSoluzione(Q, S)
  else:
    for j in range(GRANDEZZA):
      Q.append([i, j])
      if controlloSoluzione(Q):
        generaSoluzioni(Q, i+1, S)
      Q.pop()

# generaSoluzioni(Q, S)
# Parametri:
# - Q -> Insieme delle posizioni delle regine
#        (é una tabella dove ogni riga rappresenta una regina,
#        mentre ogni colonna rappresenta una delle due coordinate)
# - S -> Insieme delle soluzioni corrette
#        (é un'array tridimensionale definito in questo modo:
#        la prima dimensione definisce la soluzione considerata,
#        la seconda dimensione la regina della soluzione considerata,
#        la terza dimensione le coordinate della regina della soluzione
    considerata)
# Procedimento:
# - Genera tutte le possibili soluzioni del problema delle 8 regine e
   le salva in S
def trovaSoluzioni():
```

```
  Q = []
  S = []
  for j in range(GRANDEZZA):
    Q.append([0, j])
    generaSoluzioni(Q, 1, S)
    Q.pop()
  return S
```

3.1.8 Esempio di esecuzione

Definito il codice e il suo funzionamento, possiamo eseguirlo per generare tutte le soluzioni del problema; di seguito è riportato l'output del codice nel caso in cui la grandezza (o numero delle regine) è impostato ad 8.

```
Soluzione 1:
+-+-+-+-+-+-+-+-+
|Q| | | | | | | |
+-+-+-+-+-+-+-+-+
| | | | |Q| | | |
+-+-+-+-+-+-+-+-+
| | | | | | | |Q|
+-+-+-+-+-+-+-+-+
| | | | | |Q| | |
+-+-+-+-+-+-+-+-+
| | |Q| | | | | |
+-+-+-+-+-+-+-+-+
| | | | | | |Q| |
+-+-+-+-+-+-+-+-+
| |Q| | | | | | |
+-+-+-+-+-+-+-+-+
| | | |Q| | | | |
+-+-+-+-+-+-+-+-+
```

3.2 Implementazione in Prolog

L'implementazione riportata di seguito si divide nei seguenti punti:

- Per prima cosa andremo a definire i vari predicati sulle condizioni di validità di una soluzione.
- Successivamente andremo a definire il predicato per il controllo complessivo della soluzione.

3.2.1 Controllo della coppia

Partiamo dal predicato base, ossia quello che ci verifica che due regine non siano nella stessa colonna o riga; il predicato controlloCoppia(_/_, _/_) prende le coordinate di due regine e restituisce

semplicemente vero se le due regine non hanno nessun valore delle coordinate in comune, o falso altrimenti.

```
controlloCoppia(_/_, _/_).
```

3.2.2 Controllo della diagonale

Fatto questo possiamo passare ad un controllo più complesso, ossia verificare se due regine sono sulla stessa diagonale; per il controllo delle diagonali possiamo usare la condizione base: una coppia di regine diverse $q_i = (x_i, y_i)$ e $q_j = (x_j, y_j)$ non è sulla stessa diagonale se $x_{\neq} x_j \wedge y_i \neq y_j$. Quello che possiamo fare quindi è semplicemente verificare che le coordinate x e y delle regine siano differenti.

```
controlloDiagonale(A/B, C/D):-
    A-C =\= B-D,
    A-C =\= D-B.
```

3.2.3 Controllo delle coppie di regine

Una volta definiti i predicati base, possiamo definire un predicato che verifica tutte le coppie di regine in una lista. Questo predicato prende una lista di regine e controlla ricorsivamente che ogni coppia di regine non si trovi sulla stessa riga, colonna o diagonale.

```
controlloCoppie([A, B|T]):-
    controlloCoppia(A, B),
    controlloDiagonale(A, B),
    controlloCoppie([A|T]).
```

3.2.4 Controllo della soluzione

A questo punto non manca altro che vari il controllo della soluzione, il quale sfrutta un ragionamento per induzione: se la lista di regine contiene solo una regina, allora quella regina è in una posizione valida, mentre se sono presenti due o più regine da confrontare, allora dovrò restituire vero se la regina di riferimento e la prima della lista non possono mangiarsi e se le regine restanti con quella di riferimento non si possono mangiare.

```
controlloSoluzione([_]).

controlloSoluzione([A|T]):-
    controlloCoppie([A|T]),
    controlloSoluzione([T]).
```

3.2.5 Esempio di esecuzione

Definiti tutti i predicati, possiamo eseguirlo per verificare se una certa disposizione di regine definisce una soluzione valida.

```
? controlloSoluzione([1/6, 2/1, 3/5, 4/2, 5/8, 6/3, 7/7, 8/4]).
  true

Esecuzione 2:
? controlloSoluzione([1/2, 2/4, 3/6, 4/8, 5/3, 6/1, 7/7, 8/5]).
  true

Esecuzione 3:
? controlloSoluzione([1/5, 2/7, 3/2, 4/8, 5/5, 6/1, 7/3, 8/4]).
  false
```